Deployment Guide: Docker Compose on a VPS, Neon or Supabase, and AWS RDS

This guide deploys the User CRUD backend (Express, Prisma 7, PostgreSQL) to a real VPS using Docker Compose. It continues from the backend README, so you already have the Dockerfile and docker-compose.yml. The options at the end use a managed database (Neon, Supabase, or AWS RDS) instead of PostgreSQL in Docker.

What Changes for a Real Server

- Passwords and the database URL are no longer written in docker-compose.yml. They move into a `.env` file on the server.
- The database port 5432 is no longer opened to the internet.
- The server runs containers in the background and restarts them after a reboot.
- The frontend uses the server address in NEXT_PUBLIC_API_URL instead of localhost.

Step 1: Get a VPS

1. Rent a VPS from any provider (for example Hetzner, DigitalOcean, Lightsail, or EC2). Choose Ubuntu 24.04 or 22.04, with at least 1 GB RAM.
2. The provider gives you the server's IP address and a way to log in, either a root password or an SSH key.
3. Log in from your computer's terminal:

```
ssh root@YOUR_SERVER_IP
```

Step 2: Install Docker on the Server

On the VPS, run Docker's official convenience script, then check both tools:

```
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```

Docker Compose comes with this install, so there is nothing extra to add.

Step 3: Put Your Project on the Server

The easiest way is Git. Push backend_postgresql to a private GitHub repository, then on the server:

```
git clone https://github.com/YOUR_USER/YOUR_REPO.git
cd YOUR_REPO
```

Before pushing, make sure a `.gitignore` file contains these lines, so secrets and heavy folders stay off GitHub:

```
node_modules
.env
generated
```

The Dockerfile runs prisma generate on the server, so the generated folder does not need to be uploaded.

Step 4: Create the Password and the .env File

PostgreSQL does not give you a password. You invent it. On the server, generate a long random one:

```
openssl rand -base64 24
```

If the result contains special characters such as @, :, / or #, run `openssl rand -hex 24` instead, which gives only letters and digits and keeps the URL safe.

Now create a file named `.env` next to docker-compose.yml:

```
POSTGRES_USER=appuser
POSTGRES_PASSWORD=PASTE_YOUR_LONG_RANDOM_PASSWORD
POSTGRES_DB=usersdb
PORT=3001
```

Docker Compose reads this file automatically. The `.env` file stays on the server and is never committed to Git.

Step 5: Production docker-compose.yml

Replace the hard-coded values with variables. The backend builds its DATABASE_URL from the same values, with the host name db:

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 3s
      timeout: 3s
      retries: 10

  backend:
    build: .
    restart: unless-stopped
    ports:
      - "3001:3001"
    environment:
      DATABASE_URL:
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      PORT: ${PORT}
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

What changed from the local version:

- **No ports section on db**: PostgreSQL is reachable only by the backend inside Compose, not from the internet.
- **Variables like ${POSTGRES_PASSWORD}**: filled from the .env file.

- **restart: unless-stopped**: containers start again after a crash or a server reboot.
- **DATABASE_URL**: built from the same variables, with host name db. Because it is built by Compose, you never type the password twice.

Step 6: Start It

```
docker compose up -d --build
docker compose ps
docker compose logs -f backend
```

- `up -d --build` builds the backend and starts everything in the background.
- `ps` shows the containers and their state.
- `logs -f backend` follows the backend log. You should see "Server running on http://localhost:3001". Press Ctrl+C to stop following; the containers keep running.

Test from your own computer: open http://YOUR_SERVER_IP:3001/users in the browser. An empty array means it works.

Step 7: Open the Firewall

If the address does not respond, the server's firewall or the provider's firewall may block port 3001. On Ubuntu with ufw:

```
ufw allow OpenSSH
ufw allow 3001
ufw enable
```

Allow OpenSSH first, so you do not lock yourself out. Also check the provider's own firewall settings in its web panel.

Step 8: Point the Frontend to the Server

In the frontend folder, change .env.local to the server address:

```
NEXT_PUBLIC_API_URL=http://YOUR_SERVER_IP:3001
```

Restart npm run dev, or rebuild the frontend if it is deployed. Later, when you have a domain name, put a reverse proxy such as Caddy or Nginx in front of the backend to get HTTPS, and use https://api.yourdomain.com as the address.

Updating After a Code Change

```
git pull
docker compose up -d --build
```

The volume keeps your data, and migrate deploy applies any new migrations on startup.

Backups

A volume is not a backup. Take a dump of the database regularly:

```
docker compose exec db pg_dump -U appuser usersdb > backup.sql
```

Use your own user and database names from the .env file. Copy backup.sql off the server.

Security Checklist

- [ ] A strong random password in .env, not postgres
- [ ] .env listed in .gitignore
- [ ] No ports section on the db service
- [ ] Firewall allows only SSH and the ports you need
- [ ] Regular backups copied off the server
- [ ] Never run docker compose down -v on the server unless you want to wipe the data

Option B: Neon or Supabase Instead of PostgreSQL in Docker

With a managed database, the provider runs PostgreSQL for you. You remove the db service from Compose and give the backend the URL they provide. The provider creates the password and shows you the full connection string, so you copy it instead of inventing one. The Prisma docs confirm that Prisma ORM supports hosted PostgreSQL services such as Neon and Supabase.

How the connection works with Prisma 7

Two URLs can be involved. In Prisma 7, the connection string must be given to the driver adapter in your application code, and the Prisma CLI reads its own URL from the Prisma config file. Prisma's documentation describes this setup: a pooled DATABASE_URL for the app and a DIRECT_URL for the CLI, so migrations bypass the pooler.

Neon

1. Create a free account and a project at neon.com.
2. In the project dashboard, open the Connect panel and copy the connection string. It looks like postgresql://USER:PASSWORD@HOST/neondb?sslmode=require.
3. Neon also offers an unpooled (direct) string. Use the direct string for migrations if the pooled one gives errors.
4. Put the string in `.env` on the server as DATABASE_URL.

The adapter code in index.ts stays the same, because Neon is standard PostgreSQL and works with PrismaPg and the connection string.

Supabase

1. Create a project at supabase.com and open the Connect panel in the dashboard.
2. Supabase gives three strings: direct connection (port 5432), session pooler (port 5432), and transaction pooler (port 6543).
3. The direct connection needs IPv6 or an IPv4 add-on. If your network is IPv4-only, use the session pooler string instead.
4. For the app, use the transaction pooler string and append pgbouncer=true, as Prisma and Supabase docs advise for Supavisor. For migrations, use the direct or session string on port 5432. Do not run migrate through port 6543, because transaction pooling does not support what migrations need.

Changes in your project

In docker-compose.yml, delete the whole db service, the depends_on block, and the pgdata volume. The backend service then looks like this:

```
services:
  backend:
    build: .
    restart: unless-stopped
    ports:
      - "3001:3001"
    environment:
      DATABASE_URL: ${DATABASE_URL}
      PORT: ${PORT}
```

In the server's `.env` file:

```
DATABASE_URL=PASTE_THE_URL_FROM_NEON_OR_SUPABASE
PORT=3001
```

If the password contains special characters, they must be percentage-encoded inside the URL, according to the Prisma connection URL reference.

Run `npx prisma migrate deploy` once from your computer, or let the container's start command run it, and check that the User table appears in the provider's dashboard. If migrations hang or fail on a pooled URL, switch the migration connection to the direct string.

Option C: AWS RDS

Amazon RDS is AWS's managed PostgreSQL. Like Neon and Supabase, AWS runs the database, but you choose the password yourself when you create it, and you control who can reach it through a security group. AWS describes the basic flow in its guide to creating and connecting to a PostgreSQL DB instance.

1. In the AWS console, open RDS and choose Create database, then PostgreSQL.
2. Pick the free tier or a small instance size for learning.
3. Set a master username and a strong master password. Save the password, because AWS will not show it again.
4. Under Additional configuration, set an initial database name, for example usersdb. RDS then creates an empty database for you.
5. Choose or create a VPC security group for the database. This is the firewall that decides who may connect.
6. Wait until the status says Available, then copy the **endpoint** from the console. It is the host name of your database.

Build your DATABASE_URL from those parts, with the endpoint as the host:

```
DATABASE_URL=postgresql://MASTER_USER:MASTER_PASSWORD@YOUR_ENDPOINT:5432/usersdb?sslmode=require
```

If the password has special characters, percentage-encode them inside the URL.

Security group rule. The database accepts connections only from sources its security group allows. The safest rule is to allow port 5432 only from the security group of your application server, for example your EC2 instance. Prisma's RDS guide recommends

allowing PostgreSQL's port only from the application and a bastion host. For a quick test, you can allow port 5432 from your own IP only. Avoid allowing 5432 from any IP address, even though some tutorials show it.

Same-network note. If your backend runs on an EC2 server in the same VPC, the backend reaches the database privately, and the database does not need a public address. If you want to run migrations from your own computer, the database must be reachable from it, or you can run migrate deploy on the EC2 server instead. A common way is to run it inside the backend container, which our Dockerfile already does at startup.

Compose changes. Do the same as for Neon and Supabase: delete the db service, the depends_on block, and the pgdata volume, and pass DATABASE_URL to the backend from the server's .env file.

Before going further, test the connection from the server. If the backend cannot reach the endpoint, the cause is almost always the security group rule or the VPC, not Prisma.

Trade-offs. RDS gives you automated backups, updates, and easy scaling, and it fits well when your app already lives on AWS. It costs more than a small VPS, and it takes more clicks to set up than Neon or Supabase.

Docker PostgreSQL or managed database?

	PostgreSQL in Docker on your VPS	Neon or Supabase
Who runs the database	You	The provider
Password and URL	You create them	The provider shows them
Backups and updates	Your job	Handled by the provider
Extra cost	None beyond the VPS	Free tier, then paid plans
Best for	Learning, full control	Less maintenance

Sources

- PostgreSQL with Prisma ORM v7 (Prisma docs)
- Connect from Prisma to Neon (Neon docs)
- Prisma with Supabase (Supabase docs)
- Connection URLs reference (Prisma ORM v7)

- Creating and connecting to a PostgreSQL DB instance (AWS RDS docs)
- Configure production-ready PostgreSQL on Amazon RDS (Prisma Data Guide)